Infraestrutura de software

Implementação 4

Atenção:

- **Os arquivos de implementação deverão ser salvos dentro de um diretório cujo nome será formado pelas iniciais do e-mail em minúsculas. Exemplo: considerando que o e-mail é mla@cesar.school, o diretório será `mla`. Qualquer diretório enviado fora desse padrão será automaticamente descartado;**
- **O diretório deverá ser compactado em um arquivo `.tar` e disponibilizado para o(a) professor(a) via Google Classroom. O nome do arquivo deverá ser formado pelas iniciais do e-mail em minúsculas. Exemplo: considerando que o e-mail é mla@cesar.school, o arquivo a ser entregue será `mla.tar`. Qualquer arquivo enviado fora desse padrão será automaticamente descartado;**
- **O envio no classroom deve seguir o seguinte padrão: login.pdf e login.tar Exemplo: considerando que o e-mail é mla@cesar.school, o nome do .pdf e .tar será `mla`. Qualquer diretório enviado fora desse padrão será automaticamente descartado;**
- **Alunos com laudo pode entregar até a Sábado (07/11) até as 23:59;**
- **Identificada a cópia de qualquer questão, seja em relação a outro(a) aluno(a) ou a qualquer outra fonte, TODA a submissão será descartada.**

Sincronização do Metrô

A METROREC quer melhorar a eficiência, automatizando vagões e embarque de passageiros. Para isso, contratou você para escrever funções de sincronização que irão garantir o embarque dos vagões de forma ordenada. Nesse sistema, considere um único vagão por carro de metrô, e cada vagão e passageiro são controlados por uma thread independente. No desenvolvimento desse sistema, você precisa definir a `struct station` e demais funções descritas abaixo.

Quando um carro chega à estação e tem as portas abertas, chama-se a função

```
station_load_car(struct station *station, int count)
```

em que `count` indica quantos assentos há no carro. Essa função não retorna até o vagão estar satisfatoriamente preenchido: todos os passageiros sentados, e ou o vagão está cheio ou todos os passageiros que estavam esperando embarcaram.

Quando um passageiro chega à estação, inicialmente chama a função

```
station_wait_for_car(struct station *station)
```

que não retorna até um vagão estar na estação, ou seja, station_load_car está em execução, e que haja assentos livres no carro para o passageiro sentar-se. Uma vez que essa função retorne, o passageiro irá embarcar e ocupar um assento; esse mecanismo não é uma preocupação sua. Uma vez que o passageiro embarque, a seguinte função será chamada

```
station_on_board(struct station *station)
```

indicando ao carro que o passageiro embarcou.

Implemente o arquivo sync_car.c, que contém a declaração `struct station`, e defina as três funções acima, mais a `station_init`, que será chamada para inicializar o objeto `station`.
Ademais:

1. O programa deve ser implementado em C e ser executável em sistemas Linux, Unix ou macOS e utilizar as seguintes funções:
   - `pthread_mutex_init`
   - `pthread_mutex_lock`
   - `pthread_mutex_unlock`
   - `pthread_cond_init`
   - `pthread_cond_signal`
   - `pthread_cond_wait`

    - `pthread_cond_broadcast`
2. Não utilize semáforos nem outras primitivas de sincronização.
3. Não deve utilizar mais que um único lock em cada `struct station`.
4. Deve assumir que nunca haverá mais que um carro na estação por vez, e que todos os carros e passageiros estão indo para o mesmo destino.
5. O código deve permitir que passageiros embarquem simultaneamente. Precisa ser permitido que diversos passageiros chamem `station_wait_for_car`, e que a função retorne para cada passageiro antes de qualquer um chamar `station_on_board`.
6. O código não deve resultar em busy-wait.
7. A compilação deve ser por `Makefile`, e retornar o arquivo `sync_car` executável.

O que deve ser submetido pelo Classroom:

- Relatório em pdf, descrevendo o que foi feito, explicando, com detalhes suficientes para que eu consiga entender, como a implementação foi feita e o que foi testado. Apresente as dificuldades ao longo do processo e o que não conseguiu implementar, justificando-as. O aluno deverá seguir o formato contido no **`Guia de Relatórios`**, mas fique a vontade para incluir figuras, tabelas, gráficos, ou o que achar necessário para o entendimento.
- Link do github com readme e o relatório em pdf dentro do repositório.
- **Dentro do .tar**
    - Os arquivos com o(s) código;
    - Um breve README descrevendo todos os arquivos .c utilizados, como executar, compilar e testar o programa, além de especificar em que sistema operacional foi implementado.
    - Incluir o Makefile para compilação, limpeza dos arquivos compilados.

Rubricas

Item	Peso
README + Makefile	5%
Relatório	20%
Implementação	75%